

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное бюджетное образовательное учреждение  
высшего образования « \_\_\_\_\_ »  
Кафедра \_\_\_\_\_

**ЛАБОРАТОРНАЯ РАБОТА № 3**

по дисциплине «Основы алгоритмизации и программирование»

Эмпирический анализ временной сложности алгоритмов сортировки.

Часть 2: алгоритмы «разделяй и властвуй»  
(быстрая сортировка и сортировка слиянием)

**Вариант 6**

Направление: 38.03.05 «Бизнес-информатика»

Курс: 2

Выполнил: студент группы БИ 2-1

ФИО: Голдаева Валерия Валентиновна

Проверил: \_\_\_\_\_

2026

## 1. Цель работы и формулировка задания

Цель: изучить рекурсивные алгоритмы сортировки по принципу «разделяй и властвуй», экспериментально подтвердить сложность  $O(n \log n)$  и сравнить с квадратичными алгоритмами из ЛР № 2.

Задание варианта 6:

- Опорный элемент: первый (П)
- Разбиение: Хоара (Х)
- Слияние: восходящее (итеративное, В)
- Размеры эксперимента 2: 10 000; 20 000; 30 000; 40 000; 50 000
- Доп. задание Г6: трёхпутевое разбиение (Dutch flag) и сравнение с классическим на данных типа D

## 2. Краткие теоретические сведения

### 2.1. Парадигма «разделяй и властвуй»

Разделение  $\rightarrow$  рекурсивное решение подзадач  $\rightarrow$  комбинирование. Рекуррентность  $T(n) = 2 \cdot T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$  (основная теорема).

### 2.2. Сортировка слиянием (восходящая)

Итеративно сливает блоки длины 1, 2, 4, ... до  $n$ . Сложность  $\Theta(n \log n)$  во всех случаях. Требуется  $\Theta(n)$  дополнительной памяти. Устойчива при использовании нестрогого сравнения  $\leq$ .

### 2.3. Быстрая сортировка (Хоар, опорный = первый)

Разбиение Хоара: два указателя движутся навстречу. Опорный — первый элемент. Средний случай  $\Theta(n \log n)$ , худший (упорядоченные данные)  $\Theta(n^2)$ . Рекурсия в меньшую часть ограничивает глубину стека  $O(\log n)$ .

### 2.4. Трёхпутевое разбиение (Г6)

Алгоритм Дейкстры («голландский флаг»): элементы делятся на  $< \text{pivot}$ ,  $= \text{pivot}$ ,  $> \text{pivot}$ . На данных с большим числом повторов значительно эффективнее классического разбиения.

## 3. Блок-схемы алгоритмов

### 3.1. Быстрая сортировка

**Блок-схема: быстрая сортировка**

(Хоар, опорный = первый)

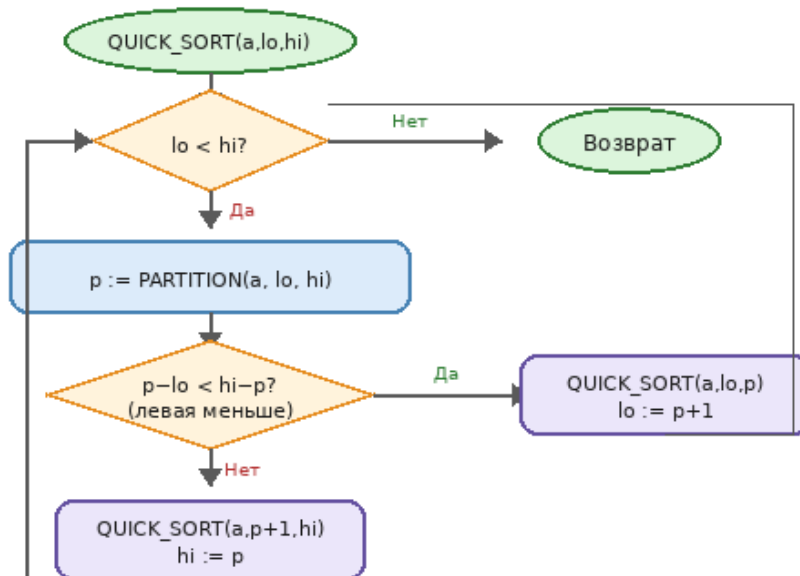


Рисунок 1 — Быстрая сортировка (рекурсия в меньшую часть)

### 3.2. Разбиение Хоара (опорный = первый)

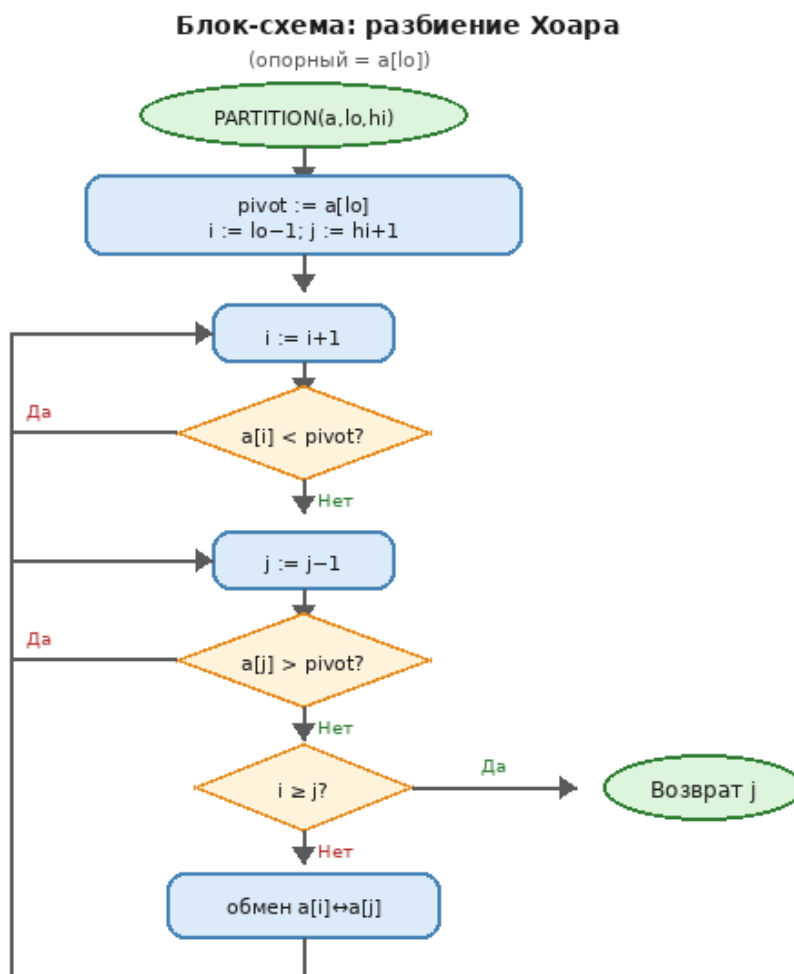


Рисунок 2 — Функция PARTITION (схема Хоара)

### 3.3. Восходящая сортировка слиянием

## Блок-схема: восходящее слияние

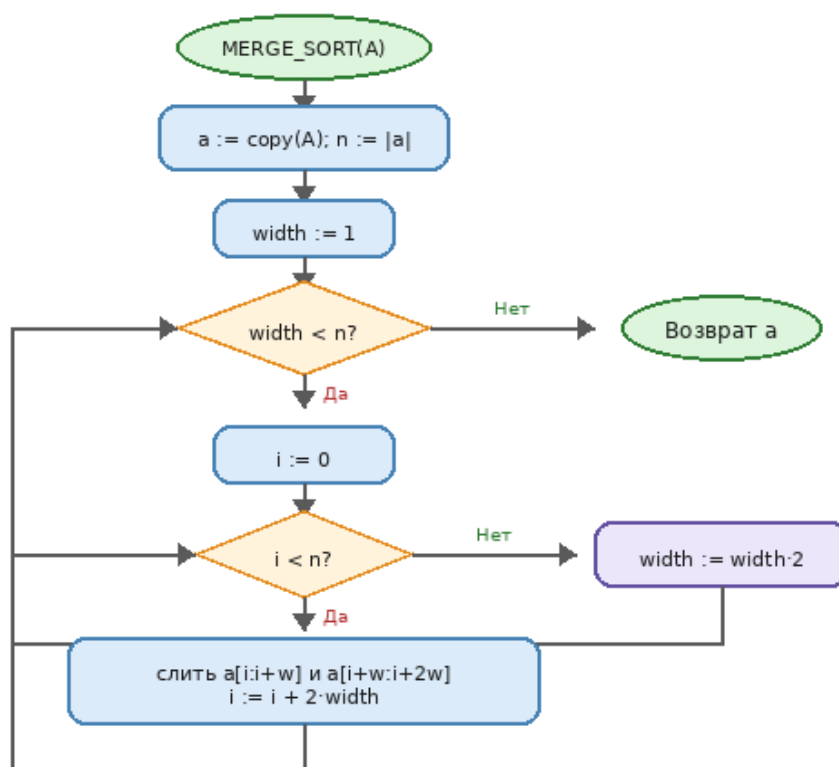


Рисунок 3 — Итеративная (bottom-up) сортировка слиянием

## 4. Листинг программы (ключевые фрагменты)

```
def quick_sort(arr):
    a = arr.copy()
    if len(a) > 1:
        _quick_sort(a, 0, len(a) - 1)
    return a

def _partition_hoare_first(a, lo, hi):
    pivot = a[lo] # опорный = первый
    i, j = lo - 1, hi + 1
    while True:
        i += 1
        while a[i] < pivot: i += 1
        j -= 1
        while a[j] > pivot: j -= 1
        if i >= j: return j
        a[i], a[j] = a[j], a[i]

def merge_sort(arr):
    # восходящая
    a = arr.copy(); n = len(a); width = 1
    while width < n:
        for i in range(0, n, 2 * width):
            left = a[i : i+width]
            right = a[i+width : i+2*width]
            a[i : i+2*width] = merge(left, right)
        width *= 2
    return a

def _quick_3way(a, lo, hi): # Dutch flag (Г6)
    if lo >= hi: return
    lt, gt, pivot, i = lo, hi, a[lo], lo
    while i <= gt:
        if a[i] < pivot:
            a[lt], a[i] = a[i], a[lt]; lt += 1; i += 1
        elif a[i] > pivot:
            a[i], a[gt] = a[gt], a[i]; gt -= 1
        else:
            i += 1
    _quick_3way(a, lo, lt-1)
    _quick_3way(a, gt+1, hi)
```

## 5. Характеристики вычислительной системы

ОС: Linux; Python 3.12; таймер: time.perf\_counter(); медиана из 3 повторов; sys.setrecursionlimit(5000).  
Для упорядоченных данных (pivot=первый) в эксперименте 3 использован  $n \leq 3000$  во избежание деградации до  $\Theta(n^2)$ .

## 6. Результаты измерений

### 6.1. Эксперимент 1 — четыре алгоритма (случайные данные), с

| n    | Пузырьком | Вставками | Быстрая | Слиянием |
|------|-----------|-----------|---------|----------|
| 500  | 0.0067    | 0.0026    | 0.0003  | 0.0006   |
| 1000 | 0.0298    | 0.0136    | 0.0008  | 0.0012   |
| 2000 | 0.1339    | 0.0576    | 0.0016  | 0.0026   |
| 4000 | 0.5459    | 0.2423    | 0.0037  | 0.0056   |

При  $n = 4000$  быстрая сортировка быстрее пузырька  $\approx$  в 150 раз, вставок  $\approx$  в 66 раз.

### 6.2. Эксперимент 2 — $O(n \log n)$ на больших массивах, с

| n      | Быстрая | Слиянием | sorted() | Слиян./Быстр. |
|--------|---------|----------|----------|---------------|
| 10 000 | 0.0101  | 0.0157   | 0.0015   | 1.55          |
| 20 000 | 0.0221  | 0.0335   | 0.0036   | 1.51          |
| 30 000 | 0.0331  | 0.0507   | 0.0045   | 1.53          |
| 40 000 | 0.0439  | 0.0729   | 0.0063   | 1.66          |
| 50 000 | 0.0570  | 0.0917   | 0.0080   | 1.61          |

### 6.3. Проверка роста $T(2n)/T(n)$

| Переход                     | Быстрая | Слиянием | Теория $O(n \log n)$ |
|-----------------------------|---------|----------|----------------------|
| 10 000 $\rightarrow$ 20 000 | 2.19    | 2.14     | $\approx 2.15$       |

Отношение  $\approx 2.1$ – $2.2$  подтверждает класс  $O(n \log n)$  (для квадратичных было бы  $\approx 4$ ).

#### 6.4. Эксперимент 3 — влияние структуры данных

| Тип данных          | n      | Быстрая | Слиянием |
|---------------------|--------|---------|----------|
| Случайные (R)       | 50 000 | 0.0597  | 0.0963   |
| Упорядоченные (S)   | 3 000  | 0.1250  | 0.0030   |
| Обратно упоряд. (V) | 3 000  | 0.1365  | 0.0028   |
| Почти упоряд. (N)   | 50 000 | 0.0803  | 0.0911   |
| Много повторов (D)  | 50 000 | 0.0547  | 0.0800   |

Важно: при опорном «первый» на упорядоченных данных быстрая сортировка деградирует до  $\Theta(n^2)$ . Уже при  $n = 3000$  она в 40 раз медленнее слияния. Поэтому для S и V взят  $n \leq 3000$ .

#### 6.5. Доп. задание Г6 — трёхпутевое разбиение на данных D

| n      | Классическая (Хоар) | Трёхпутевая (3-way) | Ускорение |
|--------|---------------------|---------------------|-----------|
| 10 000 | 0.0147              | 0.0024              | 6.1×      |
| 20 000 | 0.0179              | 0.0055              | 3.3×      |
| 50 000 | 0.0482              | 0.0158              | 3.1×      |

На данных с большим числом повторов трёхпутевое разбиение ускоряет быструю сортировку в 3–6 раз, так как элементы, равные опорному, не участвуют в дальнейшей рекурсии.

#### 6.6. Графики

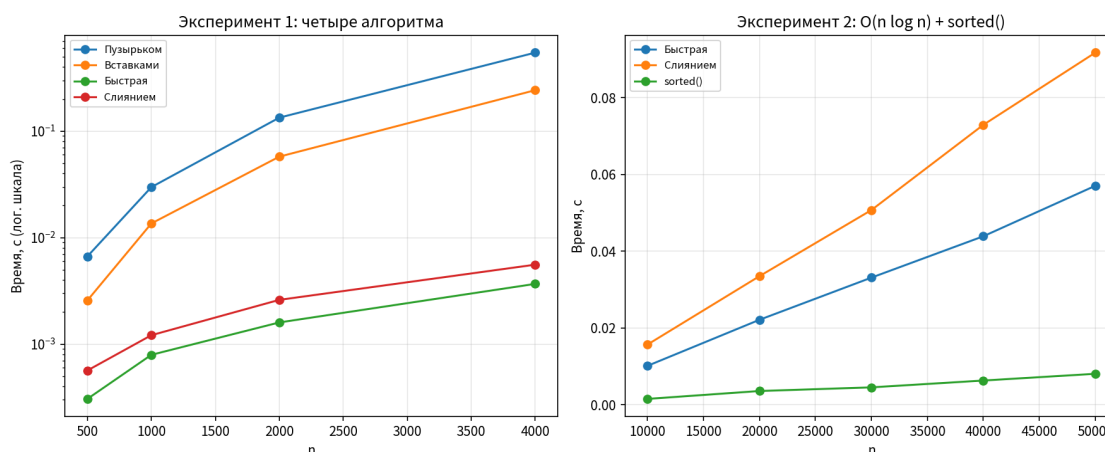


Рисунок 4 — Эксперименты 1 и 2

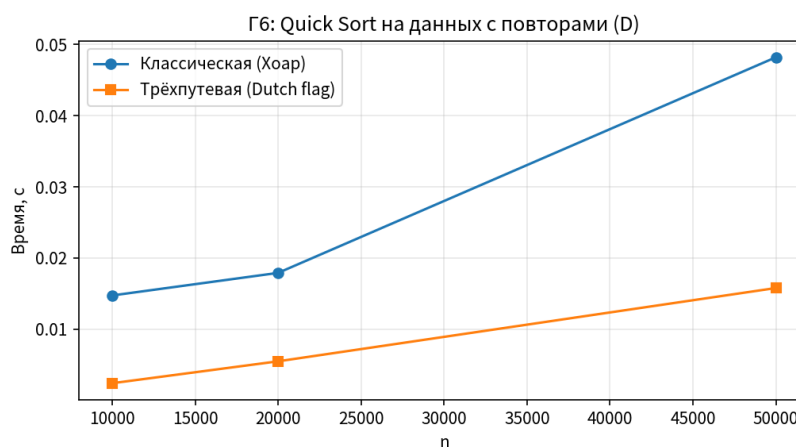


Рисунок 5 — Г6: классическая vs трёхпутевая на данных D

### 7. Сводная сравнительная таблица

| Алгоритм  | Ср. сложность | Время n=4000 | Память | Устойч. |
|-----------|---------------|--------------|--------|---------|
| Пузырьком | $\Theta(n^2)$ | 0.546 с      | $O(1)$ | Да      |

| Алгоритм  | Ср. сложность        | Время n=4000 | Память      | Устойч. |
|-----------|----------------------|--------------|-------------|---------|
| Вставками | $\Theta(n^2)$        | 0.242 с      | $O(1)$      | Да      |
| Слиянием  | $\Theta(n \log n)$   | 0.006 с      | $\Theta(n)$ | Да      |
| Быстрая   | $\Theta(n \log n)^*$ | 0.004 с      | $O(\log n)$ | Нет     |
| sorted()  | $\Theta(n \log n)$   | —            | $O(n)$      | Да      |

\* Худший случай быстрой с опорным «первый» —  $\Theta(n^2)$ .

## 8. Выводы

1. Подтверждено принципиальное различие классов сложности: при  $n = 4000$  алгоритмы  $O(n \log n)$  быстрее квадратичных в 40–150 раз.
2. Отношение  $T(2n)/T(n) \approx 2.1$ –2.2 соответствует теории  $O(n \log n)$ .
3. Быстрая сортировка на случайных данных быстрее слияния в 1.5–1.7 раза (работа на месте, лучшая локальность кэша).
4. При опорном «первый» на упорядоченных данных быстрая сортировка деградирует до  $\Theta(n^2)$  — это продемонстрировано в эксперименте 3.
5. Г6: трёхпутевое разбиение на данных с повторами даёт ускорение в 3–6 раз.
6. В прикладных системах следует использовать стандартный sorted() (Timsort на C). Собственные реализации — для учебных целей или особых ограничений.

Рекомендации: при необходимости устойчивости — слияние; при экономии памяти и случайных данных — быстрая с рандомизированным/средним опорным; при малых  $n$  — вставки; при множестве повторов — трёхпутевое разбиение.

## 9. Краткие ответы на контрольные вопросы

1. Разделение (пополам) → рекурсия → слияние упорядоченных половин.
2.  $T(n)=2T(n/2)+\Theta(n)$ . Дерево:  $\log n$  уровней  $\times \Theta(n) = \Theta(n \log n)$ .
3. Дерево решений  $\geq n!$  листьев  $\Rightarrow$  высота  $\geq \log_2(n!) = \Theta(n \log n)$ .
4. Упорядоченный массив: каждое разбиение даёт части 0 и  $n-1 \rightarrow \Theta(n^2)$ .
5. Ломуто — один указатель, больше обменов; Хоар — два указателя, меньше обменов.
6. Случайный опорный: вероятность худшего случая исчезающе мала  $\rightarrow$  ожидаемое  $\Theta(n \log n)$ .
7. Меньшая константа, in-place, лучше использует кэш, несмотря на худший случай  $\Theta(n^2)$ .
8. Рекурсия в меньшую часть, большую — циклом: глубина  $O(\log n)$ .
9. Слияние  $s \leq$  сохраняет порядок равных; обмен в quicksort его нарушает.
10. Timsort: поиск runs + слияние + вставки для коротких. Вставки оптимальны на почти упорядоченных.
11. Внешняя сортировка (многофазное слияние) — данные не помещаются в ОП.

— Конец отчёта —